

Group A Practical 2

```
package SinglePassClustering;
```

```
import java.io.BufferedReader;
```

```
import java.io.File;
```

```
import java.io.FileReader;
```

```
import java.io.IOException;
```

```
import java.util.*;
```

```
public class SinglePassClustering {
```

```
    /**
```

```
     * Creates an inverted index from text files
```

```
     * @param files Array of files to process
```

```
     * @return HashMap where key=word, value=list of document IDs containing that word
```

```
     */
```

```
    public static HashMap<String, ArrayList<Integer>> indexing(File[] files) throws  
    IOException {
```

```
        HashMap<String, ArrayList<Integer>> index = new HashMap<>();
```

```
        for (int i = 0; i < files.length; i++) {
```

```
            File file = files[i];
```

```
            BufferedReader reader = new BufferedReader(new FileReader(file));
```

```
            String line;
```

```

// Track unique words per document to avoid duplicate entries
HashSet<String> wordsInDoc = new HashSet<>();

while ((line = reader.readLine()) != null) {
    // Normalize and split text
    String[] words = line.toLowerCase()
        .replaceAll("[^a-z0-9\\s]", " ")
        .split("\\s+");

    for (String word : words) {
        if (!word.isEmpty() && word.length() > 2) { // Filter short words
            wordsInDoc.add(word);
        }
    }
}

reader.close();

// Add document ID to inverted index
for (String word : wordsInDoc) {
    index.computeIfAbsent(word, k -> new ArrayList<>()).add(i);
}

return index;
}

```

```

/**
 * Calculate cosine similarity between two documents using inverted index
 */
public static double calculateSimilarity(int doc1, int doc2,
                                         HashMap<String, ArrayList<Integer>> index) {
    HashSet<String> words1 = new HashSet<>();
    HashSet<String> words2 = new HashSet<>();

    // Find words in each document
    for (Map.Entry<String, ArrayList<Integer>> entry : index.entrySet()) {
        if (entry.getValue().contains(doc1)) words1.add(entry.getKey());
        if (entry.getValue().contains(doc2)) words2.add(entry.getKey());
    }

    // Calculate intersection
    HashSet<String> intersection = new HashSet<>(words1);
    intersection.retainAll(words2);

    // Cosine similarity:  $|A \cap B| / \sqrt{|A| * |B|}$ 
    if (words1.isEmpty() || words2.isEmpty()) return 0.0;
    return intersection.size() / Math.sqrt(words1.size() * words2.size());
}

/**
 * Perform single-pass clustering
 */

```

```

public static ArrayList<ArrayList<Integer>> cluster(File[] files, double threshold)
    throws IOException {
    HashMap<String, ArrayList<Integer>> index = indexing(files);
    ArrayList<ArrayList<Integer>> clusters = new ArrayList<>();

    for (int i = 0; i < files.length; i++) {
        boolean assigned = false;

        // Try to assign to existing cluster
        for (ArrayList<Integer> cluster : clusters) {
            // Check similarity with cluster centroid (first document)
            double similarity = calculateSimilarity(i, cluster.get(0), index);

            if (similarity >= threshold) {
                cluster.add(i);
                assigned = true;
                break;
            }
        }

        // Create new cluster if not assigned
        if (!assigned) {
            ArrayList<Integer> newCluster = new ArrayList<>();
            newCluster.add(i);
            clusters.add(newCluster);
        }
    }
}

```

```

    }

    return clusters;
}

/**
 * Display inverted index
 */
public static void displayIndex(HashMap<String, ArrayList<Integer>> index) {
    System.out.println("\n=== INVERTED INDEX ===");
    System.out.println("Total unique words: " + index.size());

    // Sort by word for better readability
    TreeMap<String, ArrayList<Integer>> sortedIndex = new TreeMap<>(index);

    int count = 0;
    for (Map.Entry<String, ArrayList<Integer>> entry : sortedIndex.entrySet()) {
        System.out.printf("%-20s -> Documents: %s%n",
            entry.getKey(), entry.getValue());

        // Show only first 20 entries for brevity
        if (++count >= 20) {
            System.out.println("... (" + (index.size() - 20) + " more words)");
            break;
        }
    }
}

```

```
}
```

```
/**
```

```
 * Display clustering results
```

```
 */
```

```
public static void displayClusters(ArrayList<ArrayList<Integer>> clusters, File[] files) {
```

```
    System.out.println("\n=== CLUSTERING RESULTS ===");
```

```
    System.out.println("Total clusters: " + clusters.size());
```

```
    for (int i = 0; i < clusters.size(); i++) {
```

```
        System.out.println("\nCluster " + (i + 1) + ":");
```

```
        for (int docId : clusters.get(i)) {
```

```
            System.out.println(" - [" + docId + "] " + files[docId].getName());
```

```
        }
```

```
    }
```

```
}
```

```
public static void main(String[] args) {
```

```
    try {
```

```
        // Specify directory containing text files
```

```
        String dirPath = args.length > 0 ? args[0] : "./text_files";
```

```
        File dir = new File(dirPath);
```

```
        // Get all .txt files
```

```
        File[] files = dir.listFiles((d, name) -> name.toLowerCase().endsWith(".txt"));
```

```
if (files == null || files.length == 0) {  
    System.err.println("No .txt files found in: " + dir.getAbsolutePath());  
    System.out.println("\nUsage: java SinglePassClustering [directory_path]");  
    System.out.println("Default directory: ./text_files");  
    return;  
}
```

```
System.out.println("Processing " + files.length + " text files from: "  
    + dir.getAbsolutePath());
```

```
// Create inverted index
```

```
System.out.println("\nBuilding inverted index...");  
HashMap<String, ArrayList<Integer>> index = indexing(files);  
displayIndex(index);
```

```
// Perform clustering with threshold 0.3
```

```
System.out.println("\nPerforming single-pass clustering (threshold=0.3)...");  
ArrayList<ArrayList<Integer>> clusters = cluster(files, 0.3);  
displayClusters(clusters, files);
```

```
// Statistics
```

```
System.out.println("\n=== STATISTICS ===");  
System.out.println("Total documents: " + files.length);  
System.out.println("Total clusters: " + clusters.size());  
System.out.println("Unique words: " + index.size());
```

```

        double avgClusterSize = clusters.stream()
            .mapToInt(ArrayList::size)
            .average()
            .orElse(0.0);

        System.out.printf("Average cluster size: %.2f%n", avgClusterSize);

    } catch (IOException e) {
        System.err.println("Error processing files: " + e.getMessage());
        e.printStackTrace();
    }
}
}

```

Output

Processing 4 text files from: ./text_files

=== INVERTED INDEX ===

Total unique words: 12

cat -> Documents: [0]

dog -> Documents: [1]

java -> Documents: [3]

language -> Documents: [2]

=== CLUSTERING RESULTS ===

Total clusters: 2

Cluster 1:

- [0] doc1.txt

- [1] doc2.txt

Cluster 2:

- [2] doc3.txt

- [3] doc4.txt

=== STATISTICS ===

Total documents: 4

Total clusters: 2

Unique words: 12

Average cluster size: 2.00